Boring-NixOS

K19G

2026-09-08

Table of contents

Boring NixOS

Boring NixOS

A direct, practical guide to configuring infrastructure that builds identically every time: **deterministic, immutable, and refreshingly boring**.

Ambiguity is the enemy of reliability. When machines mutate silently through ad-hoc commands, builds drift, onboarding breaks, and outages follow. The boring default is to define the full closure in code — from developer compilers to multi-region cloud clusters — so every environment evaluates identically six months later.

Baseline: Nix **2.24+** · NixOS **24.11+** · Flakes enabled. Standalone and self-contained. You do not need any other title in this library.

Who this is for

- Engineers, platform developers, and SREs who want systems that build identically on a laptop, CI runner, and bare-metal server.
- Developers looking for instant, clean project environments without container lag or host library pollution.
- Teams moving away from fragile setup scripts and mutable system administration toward pure declarative configuration.

You do not need prior experience with Nix or functional programming. A terminal, a standard text editor, and basic Linux familiarity are all you need.

What "boring" means here

Nix is a purely functional package manager and domain-specific language. Applied with discipline, it eliminates entire categories of operational bugs: conflicting shared libraries, broken symlinks, broken OS upgrades, and undocumented host drift.

When over-complicated, it can become an impenetrable maze of nested overlays and tangled macros.

Boring NixOS teaches clear, sustainable defaults: - Plain, readable expressions over clever metaprogramming. - Explicit inputs pinned by cryptographic lockfiles. - Modular, auditable NixOS system definitions that scale without surprise.

How to read

1. Read the chapter concept and mental model.
2. Type the expression into its designated file (`default.nix`, `shell.nix`, `flake.nix`, or `configuration.nix`).
3. Run the exact build or evaluation command.
4. Experiment with the **Try this** exercises to verify how the store and evaluator behave.

The chapters are sequenced progressively. You can jump directly to devShells, NixOS services, or CI caching once you know the language basics.

How examples work

Every code listing is a complete, runnable expression or file. Nothing is an incomplete fragment left for you to guess.

```nix
# default.nix
let
  message = "reproducible desk online";
in
message
```

```
nix-instantiate --eval default.nix
```

```
"reproducible desk online"
```

A recurring scenario — a small team infrastructure desk (workstations, devShells, microservices, deployment pipelines) — connects the exercises so each technique solves a practical engineering challenge.

Map of the book

Part	Folder	What you leave with
0	`00-introduction`	The cost of non-reproducible systems, why imperative setups drift, and the immutable alternative
1	`01-nix-fundamentals`	The `/nix/store` layout, cryptographic hashes, dependency DAGs, garbage collection, and raw derivations

Part	Folder	What you leave with
2	`02-nix-language`	Primitive types, `let ... in` bindings, lambda functions, attribute sets, lists, and modular imports
3	`03-dev-environments`	Instant project toolchains with `mkShell`, multi-language runtimes, Flakes, Home Manager, and replacing heavy local containers
4	`04-packages-and-overlays`	The package registry, managing pinned Flake inputs, patching packages with overlays, and project dependency trees
5	`05-building-from-source`	Packaging software from source: C/CMake, Rust Cargo, Python, Go, Node.js, and multi-architecture cross-compiles
6	`06-nixos-system`	The NixOS module system, declarative machine definitions, systemd services, firewalls, users, and hardware profiles
7	`07-home-manager`	User-space configuration: declarative dotfiles, shell environments, editor configs, and unifying user space with NixOS
8	`08-cicd`	Continuous integration that builds once and caches everywhere: binary caches, GitHub Actions, GitLab CI, and secrets
9	`09-containers-k8s`	Minimal rootless container images with `dockerTools`, declarative Kubernetes manifests, Helm generation, and GitOps workflows
10	`10-iac`	Declarative cloud infrastructure: managing Terraform and OpenTofu providers, Ansible provisioning, and multi-cloud patterns

Part	Folder	What you leave with
11	`11-secrets-security`	Safe secret distribution in declarative systems: SOPS-nix, Age keys, HashiCorp Vault, and OS security hardening
12	`12-platform-engineering`	Internal developer platforms, monorepo dependency graphs, fleet upgrades, build tuning, and production checklists
99	`99-appendices`	Glossary of terms and command cheat sheet

What this book is not

- Not an exhaustive reference manual of all 100,000+ packages in `nixpkgs`.
- Not an exercise in esoteric functional programming puzzles.
- Not a guide to maintaining legacy channels and unpinned channels.

It is a concrete guide to writing reproducible systems that remain easy to understand, maintain, and operate.

Introduction

The Reproducibility Crisis

The Reproducibility Crisis

Software engineering has spent decades tolerating ambient state: hidden dependencies, global machine mutations, and "works on my machine" excuses. The boring default of this book is: **every development environment, dependency, and production system must be a pure, declarative, reproducible expression from zero**.

Mental model

Traditional operations rely on imperative mutation. You provision a virtual machine or container, run a sequence of `apt-get`, `curl | bash`, and `pip install` commands, and hope that: 1. Upstream package repositories haven't updated or removed files. 2. Network connections remain uninterrupted mid-install. 3. System state matches the exact sequence of commands previously tested.

When any of these fail, you get **Heisenbugs**: intermittent build failures, subtle ABI mismatches, and production outages that disappear when you attempt to isolate them.

```
Traditional Imperative Model:
  Base OS -> [Step 1: apt-get] -> [Step 2: pip install] -> [Step 3: git clone] -> ??? (Unknown

Nix Declarative Model:
  Inputs (Source + Hashes) -> Pure Derivation -> Immutable /nix/store Path (Deterministic)
```

Nix replaces stateful mutation with pure mathematical evaluation: given the exact same source code and inputs, it always yields the exact same byte-for-byte system closure.

Worked examples

Case 1: Detecting Ambient Host Contamination

Save as `test_environment.sh`. Inspect how standard shell scripts unknowingly depend on host binaries and paths.

```
# test_environment.sh
#!/usr/bin/env bash
set -euo pipefail

echo "Inspecting system path dependencies:"
which python3 || echo "python3 missing"
```

```
which node || echo "node missing"
which gcc || echo "gcc missing"
```

Run:

```
bash test_environment.sh
```

Output (on a typical developer workstation):

```
Inspecting system path dependencies:
/usr/bin/python3
/home/user/.nvm/versions/node/v20.0.0/bin/node
/usr/bin/gcc
```

Notice how three completely different tools live in three different unpinned locations (`/usr/bin`, user home directories, global OS paths). If a colleague runs the exact same script on a fresh laptop, it fails immediately.

Case 2: Purity and Isolated Evaluation

Save as `pure_check.nix`. A minimal Nix expression evaluated in isolation.

```
# pure_check.nix
let
  system = builtins.currentSystem;
  message = "No ambient host contamination allowed";
in
{
  inherit system message;
}
```

Run:

```
nix-instantiate --eval pure_check.nix
```

Output:

```
{ message = "No ambient host contamination allowed"; system = "x86_64-linux"; }
```

Evaluating this expression does not query the internet, look at `/usr/local`, or depend on environment variables.

Case 3: Hermetic Sandboxing in Action

Nix builds run in sandboxed namespaces where network access and non-declared filesystem paths are inaccessible.

Save as `sandbox_check.nix`:

```
# sandbox_check.nix
derivation {
  name = "sandbox-proof";
  system = builtins.currentSystem;
  builder = "/bin/sh";
  args = [
    "-c"
    "echo 'Built inside isolated namespace' > $out"
  ];
}
```

Run:

```
nix-build sandbox_check.nix
```

Output:

```
these 1 derivations will be built:
  /nix/store/10h9v...-sandbox-proof.drv
building '/nix/store/10h9v...-sandbox-proof.drv'...
/nix/store/v73ka...-sandbox-proof
```

Inspect output:

```
cat result
```

Output:

```
Built inside isolated namespace
```

The output was generated purely from the declared inputs.

The trap

Relying on “Golden Images” (pre-baked AMIs, Docker images without lockfiles, or manual server setup steps documented in internal wikis).

```
Wiki page: "Run these 14 commands to set up the desk service..."
```

By week three, one PPA repository goes offline, a patch release bumps a minor dependency, and onboarding is broken for every new hire.

The fix: Check in code that defines the environment completely down to cryptographic hashes. If it builds once, it builds forever.

The boring rule

- Never mutate system state in-place with ad-hoc shell commands.
- Pin all toolchain and library dependencies via cryptographic hashes.
- Eliminate dependency on the host's ambient `/usr/bin` and `$PATH`.
- Every environment must be reproducible from a fresh Git checkout.

Try this

1. Run `env` in your current shell and note how many variables point to user home directories or mutable local paths.
2. In `pure_check.nix`, add a field `timestamp = builtins.currentTime;` and observe how pure evaluation restricts non-deterministic built-ins.

Nix Fundamentals

The Store Model and Content-Addressable Paths

The Store Model and Content-Addressable Paths

In a traditional operating system, files are scattered across mutable locations like `/usr/bin`, `/usr/lib`, and `/etc`. The boring default of Nix is: **every file, binary, and library lives in an immutable, isolated path under `/nix/store`, completely eliminating file collision**.

Mental model

The Nix store is the foundation of reproducible systems. Every entry follows a strict naming convention:

`/nix/store/<hash>-<name>-<version>`

For example:

`/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0`

1. **The Hash (32 characters base32):** Computed from the cryptographic closure of all build inputs (source code, compiler, compiler flags, and dependencies).
2. **Immutable Filesystem:** Store directories are marked read-only immediately after build completion. Even root cannot accidentally overwrite them without explicitly mounting with write permissions or breaking store integrity.
3. **No Collision:** You can install Python 3.10, Python 3.11, and Python 3.12 simultaneously on the same machine. None of them collide because each has its own independent hash path.
4. **Input-Addressed vs Content-Addressed:** Historically, paths were input-addressed (derived from the build inputs). Content-addressable paths hash the final built artifact itself, enabling even deeper cache sharing.

Worked examples

Case 1: Inspecting the Store Directory Anatomy

Examine a store path directly in your shell.

Run:

```
ls -ld $(which nix)
```

Output:

```
-r-xr-xr-x 1 root root 15728640 Jan 1 1970 /nix/store/7vqw9388...-nix-2.24.1/bin/nix
```

Notice two critical details: - Permissions are `-r-xr-xr-x` (read and execute only; no write permission). - The timestamp is standardized to `Jan 1 1970` (the Unix epoch) to preserve cryptographic bit-for-bit build reproducibility across machines and time zones.

Case 2: Store Path Dissection

Save as `dissect_path.sh`. A script that analyzes the parts of a Nix store path.

```
# dissect_path.sh
#!/usr/bin/env bash
set -euo pipefail

path="/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0"
basename="$(basename "$path")"

hash="${basename:0:32}"
pkgname="${basename:33}"

echo "Full store path: $path"
echo "Cryptographic hash: $hash"
echo "Package name & version: $pkgname"
```

Run:

```
bash dissect_path.sh
```

Output:

```
Full store path: /nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0
Cryptographic hash: z1k947f6y788sfaivs931h26hsvb3506
Package name & version: ripgrep-14.1.0
```

The 32-character hash guarantees uniqueness. If ripgrep is rebuilt against a newer glibc, its hash changes, leaving the existing path intact.

Case 3: Verifying Store Path Immutability

Attempting to mutate an existing file inside `/nix/store` is rejected by the kernel.

Run:

```
touch $(which nix) 2>&1 || true
```

Output:

```
touch: cannot touch '/nix/store/...-nix-2.24.1/bin/nix': Permission denied
```

Files inside `/nix/store` cannot be modified in place. Upgrades and reconfigurations always produce new paths.

The trap

Treating `/nix/store` like an ordinary mutable directory and trying to `sudo chmod +w` or manually edit files inside it. Doing so invalidates the cryptographic checksums tracked by the Nix database and corrupts your system state.

The fix: If you need to change a package or script, edit its Nix expression and rebuild. Nix creates a new clean store path without touching the old one.

The boring rule

- Never manually edit, touch, or delete files directly in `/nix/store`.
- Rely on symlinks (`result`, `/run/current-system`, `~/.nix-profile`) to point to active store paths.
- Let garbage collection (`nix-collect-garbage`) handle disk space cleanup safely.

Try this

1. Run `ls /nix/store | head -n 5` and observe the naming pattern of packages installed on your system.
2. Find the size of any package in your store using `du -sh <store-path>`.

How Hashes Create Deterministic Dependencies

How Hashes Create Deterministic Dependencies

Dynamic linking in traditional Linux is based on file names like `libc.so.6`. If that file is updated globally, every application on the host changes behavior simultaneously. The boring default of Nix is: **embed the exact cryptographic hash of every required dependency into the binary's runtime header (ELF RPATH)**.

Mental model

How does a binary built with Nix know where its dependencies live? When Nix builds an application, it records the exact store paths of its libraries directly into the binary's ELF header under `RUNPATH` / `RPATH`:

```
Traditional Binary:
  RPATH: /usr/lib:/lib
  -> Looks for whatever libc.so is installed in global directories.

Nix Binary:
  RPATH: /nix/store/7r44p...-glibc-2.39/lib
  -> Hardcoded, deterministic pointer to the exact version used at build time.
```

Because the path points directly to `/nix/store/<hash>-glibc`, upgrading or installing another glibc elsewhere on the machine has zero effect on this binary.

Worked examples

Case 1: Inspecting ELF RPATH

Save as `check_rpath.sh`. Use `patchelf` or `readelf` to verify that runtime library paths are locked to explicit store paths.

```
# check_rpath.sh
#!/usr/bin/env bash
set -euo pipefail

binary="$(which rg)"
readelf -d "$binary" | grep -E '(RPATH|RUNPATH)' || echo "Static or system binary"
```

Run:

```
bash check_rpath.sh
```

Output:

```
0x000000000000001d (RUNPATH)            Library runpath: [/nix/store/7r44p12l4b72j9m9l09k96ygh
```

The dynamic linker is explicitly told to look in that specific store path, never in global directories.

Case 2: Querying Direct Dependencies with nix-store

Find the direct dependencies of an executable.

Run:

```
nix-store -q --references $(which rg)
```

Output:

```
/nix/store/7r44p12l4b72j9m9l09k96yghz02f8n7-glibc-2.39
/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0
```

There are no mystery dependencies; every needed path is listed.

The trap

Setting `LD_LIBRARY_PATH` globally in your `~/.bashrc` to "fix" a missing library. This overrides the explicit `RPATH` in binaries and reintroduces ambient host contamination across all running processes.

The fix: Let Nix derivations and development shells set library paths via `buildInputs`. Never export global `LD_LIBRARY_PATH` variables.

The boring rule

- Every library dependency is referenced by its immutable `/nix/store` hash path.
- Binaries use `RUNPATH` to ensure deterministic dynamic linking.
- Avoid global environment variable overrides (`LD_LIBRARY_PATH`, `PYTHONPATH`).

Try this

1. Run `ldd $(which bash)` on a Nix-managed system and examine where each shared library points.

Dependency Graphs and the Nix Database

Dependency Graphs and the Nix Database

Every software artifact in Nix is a node in an immutable directed acyclic graph (DAG). The boring default is: **inspect dependency closures with `nix-store -q` and trust the local SQLite database (`/nix/var/nix/db/db.sqlite`) to track all package registrations accurately**.

Mental model

Nix distinguishes between two kinds of dependencies:

1. **Build-time dependencies (the derivation closure):** Everything required to compile the package from source: compilers, build tools, header files, test runners, and documentation generators.
2. **Runtime dependencies (the runtime closure):** Only the files and libraries strictly required for the final binary to execute.

When a package is installed, Nix scans the built files for references to any build-time store paths. Only referenced paths are included in the runtime closure. Compilers, build scripts, and header files are excluded automatically, keeping production images lightweight.

All store registrations, valid paths, and reference relationships are recorded in `/nix/var/nix/db/db.sqlite`.

Worked examples

Case 1: Querying the Full Runtime Closure

Print the complete list of all transitive dependencies required to run an application.

Run:

```
nix-store -q --requisites $(which rg)
```

Output:

```
/nix/store/7r44p12l4b72j9m9l09k96yghz02f8n7-glibc-2.39
/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0
```

`--requisites` calculates the transitive closure: everything needed to copy this program onto another bare machine and run it immediately.

Case 2: Querying Reverse Dependencies (Referrers)

Find out which packages on your system depend on a specific library.

Run:

```
nix-store -q --referrers /nix/store/7r44p12l4b72j9m9l09k96yghz02f8n7-glibc-2.39 | head -n 3
```

Output:

```
/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0
/nix/store/v3n8...-bash-5.2p26
/nix/store/a81c...-coreutils-9.5
```

This prevents accidental deletion of shared libraries that other active packages depend on.

The trap

Confusing build-time dependencies with runtime dependencies. For example, assuming that adding `gcc` to your build will cause `gcc` to be shipped in your final production container image. If the compiled binary does not reference `gcc`'s path, Nix excludes it from the runtime closure.

The boring rule

- Use `nix-store -q --requisites` to see the true deployment size and closure of any package.
- Use `nix-store -q --referrers` to see what is keeping an old package from being garbage collected.
- Never manually edit the Nix SQLite database.

Try this

1. Run `nix-store -q --tree $(which nix)` and observe the hierarchical closure graph printed in your terminal.

Garbage Collection and Space Management

Garbage Collection and Space Management

Because Nix never overwrites existing packages, disk usage in `/nix/store` naturally grows over time. The boring default is: **use garbage collector roots (`gcroots`) to preserve active environments, run `nix-collect-garbage -d` to clean unreferenced paths, and deduplicate files with `nix-store --optimise`**.

Mental model

Nix uses tracing garbage collection analogous to programming languages like Go or Java:

1. **GC Roots:** Pointers that tell the garbage collector "do not delete this path". GC roots include:
   - System generations (e.g., `/nix/var/nix/profiles/system-42-link`)
   - User profile generations (`~/.nix-profile`)
   - Active `result` symlinks created by `nix-build` in your working directories
2. **Reachable Paths:** Any store path reachable by traversing the dependency DAG from any GC root is preserved.
3. **Dead Paths:** Any store path with zero references from any GC root is considered garbage and eligible for deletion.
4. **Hard Link Optimization:** When different packages contain identical files, `nix-store --optimise` replaces redundant copies with hard links, saving significant gigabytes.

Worked examples

Case 1: Inspecting Active GC Roots

List registered roots protecting packages from garbage collection.

Run:

```
ls -l /nix/var/nix/gcroots/auto | head -n 3
```

Output:

```
lrwxrwxrwx 1 root root 28 Sep  8 10:00 /nix/var/nix/gcroots/auto/abc123... -> /home/user/projec
```

Every active build result symlink is registered as a root so that running garbage collection during an active build does not delete your current work.

Case 2: Running Safe Garbage Collection

Run garbage collection without deleting older generation rollbacks.

Run:

```
nix-collect-garbage
```

Output:

```
finding garbage collector roots...
deleting garbage...
deleting '/nix/store/7vqw9388...-unused-pkg'
deleting '/nix/store/m6a1p12l...-temporary-build'
14 store paths deleted, 142.50 MiB freed
```

This cleans only unreferenced, orphaned files while keeping previous bootable system generations intact.

Case 3: Deleting Old Generations and Full Purge

Delete older system and profile rollbacks, then reclaim all freed space.

Run:

```
nix-collect-garbage -d
```

Output:

```
removing old generations...
finding garbage collector roots...
deleting garbage...
124 store paths deleted, 2.10 GiB freed
```

`-d` (delete old generations) removes all previous history, keeping only the currently running generation.

Case 4: Hard Link Deduplication

Reclaim space by deduplicating identical files across all installed store paths.

Run:

```
nix-store --optimise
```

Output:

```
hashing files in /nix/store...
31420 files linked, 850.40 MiB freed
```

Identical files across different package versions (such as licenses, header files, or shared assets) now share identical physical inodes.

The trap

Running `rm -rf /nix/store/<some-path>` directly with `sudo`. Doing so corrupts the Nix SQLite registration database and will break other packages that depend on that path.

The fix: Always use `nix-collect-garbage`. To delete a package, delete the root or profile symlink pointing to it, and run `nix-collect-garbage`.

The boring rule

- Always delete packages via `nix-collect-garbage`, never with manual `rm`.
- Run `nix-collect-garbage -d` when disk space is constrained and you no longer need rollback to older generations.
- Enable automatic weekly GC and store optimization in your NixOS configuration (`nix.gc.automatic = true;`).

Try this

1. Build a local derivation with `nix-build`, verify the `result` symlink exists, and test that `nix-collect-garbage` preserves it.
2. Delete the `result` symlink (`rm result`) and re-run `nix-collect-garbage` to observe it being deleted.

Your First Nix Expressions

Your First Nix Expressions

A Nix expression is code that evaluates to a package or configuration value. The boring default is: **understand the low-level `derivation` primitive first, then build clean expressions with `nix-build`.**

Mental model

At the lowest level, Nix builds packages using the built-in function `derivation`. A derivation requires three mandatory attributes: 1. `name`: A string identifier. 2. `system`: The target architecture string (e.g. `"x86_64-linux"`, `"aarch64-darwin"`). 3. `builder`: The path to the executable responsible for executing the build (usually `/bin/sh` or bash).

When evaluated, a `.nix` file generates a `.drv` (derivation specification file) in `/nix/store`. When built with `nix-build`, Nix runs the builder inside an isolated sandbox and captures the file or directory written to `$out`.

Worked examples

Case 1: Minimal Self-Contained Derivation

Save as `minimal.nix`. A derivation requiring zero external packages.

```
# minimal.nix
derivation {
  name = "desk-banner";
  system = builtins.currentSystem;
  builder = "/bin/sh";
  args = [
    "-c"
    "echo 'Desk System Online' > $out"
  ];
}
```

Run:

```
nix-build minimal.nix
```

Output:

```
these 1 derivations will be built:
  /nix/store/7vqw93...-desk-banner.drv
building '/nix/store/7vqw93...-desk-banner.drv'...
/nix/store/3la841...-desk-banner
```

Check the generated output:

```
cat result
```

Output:

```
Desk System Online
```

`$out` is the destination path allocated in `/nix/store` for this build.

Case 2: Multi-File Build in a Derivation

Save as `multifile.nix`. Create a directory structure with multiple output files.

```
# multifile.nix
derivation {
  name = "desk-assets";
  system = builtins.currentSystem;
  builder = "/bin/sh";
  args = [
    "-c"
    ''
      mkdir -p $out/bin $out/share
      echo '#!/bin/sh' > $out/bin/desk-status
      echo 'echo status: ready' >> $out/bin/desk-status
      chmod +x $out/bin/desk-status
      echo 'Desk version 1.0' > $out/share/version.txt
    ''
  ];
}
```

Run:

```
nix-build multifile.nix
```

Output:

```
these 1 derivations will be built:
  /nix/store/41p9...-desk-assets.drv
building '/nix/store/41p9...-desk-assets.drv'...
/nix/store/99ka...-desk-assets
```

Test the generated script:

```
./result/bin/desk-status
cat ./result/share/version.txt
```

Output:

```
status: ready
Desk version 1.0
```

`$out` can be either a single file or a directory containing a complete application structure (`bin/`, `lib/`, `share/`).

The trap

Hardcoding mutable user directories or environment variables inside derivations. Nix derivations cannot access `$HOME` or external files outside the declared source tree.

The fix: Always write outputs to `$out`, and pass all input sources explicitly.

The boring rule

- A derivation is a pure function that writes its output to `$out`.
- `nix-instantiate` creates `.drv` files; `nix-build` executes them to produce store paths.
- Avoid low-level raw derivations in daily work; use `stdenv.mkDerivation` or `pkgs.mkShell` which wrap this primitive with sensible defaults.

Try this

1. In `minimal.nix`, add an environment variable attribute `greeting = "Front Desk Active";` and reference `$greeting` inside the build script.
2. Inspect the `.drv` file generated by `nix-instantiate minimal.nix` with `nix show-derivation`.

Nix Language

Expressions, Values, and Types

Expressions, Values, and Types

Nix is not an imperative scripting language; it is a **pure, functional, lazily evaluated expression language**. The boring default is: **master the primitive types (integers, strings, booleans, paths, and null) and let lazy evaluation calculate only the values your build actually requires**.

Mental model

Everything in Nix is an expression that evaluates to a value. There are no statements, loops, or mutable variables.

1. **Primitives:**
   - Numbers: Integers (`42`), Floats (`3.14`)
   - Booleans: `true`, `false`
   - Strings: Single-line (`"hello"`), Multi-line (`''line 1\nline 2''`)
   - Paths: Unquoted with slashes (`./config.nix`, `/etc/passwd`)
   - Null: `null`
2. **String Interpolation:** Embedded expressions inside `"${expr}"` or `''${expr}''`.
3. **No Variables:** You cannot write `x = 1; x = 2;`. Identifiers are bound once and are immutable.

Worked examples

Case 1: Evaluating Primitive Literals

Save as `primitives.nix`.

```
# primitives.nix
{
  port = 8080;
  rate = 12.5;
  enabled = true;
  description = "Desk order processing node";
  fallback = null;
  configPath = ./config.nix;
}
```

Run:

```
nix-instantiate --eval primitives.nix
```

Output:

```
{ configPath = /home/user/project/config.nix; description = "Desk order processing node"; enabl
```

Notice that `./config.nix` evaluated directly to an absolute filesystem path.

Case 2: Multi-Line Strings and Indentation Stripping

Save as `multiline.nix`. Nix multi-line strings automatically strip leading common indentation.

```
# multiline.nix
let
  serviceName = "desk-api";
  script = ''
    #!/bin/sh
    echo "Starting ${serviceName}..."
    exec ${serviceName} --port 8080
  '';
in
script
```

Run:

```
nix-instantiate --eval multiline.nix
```

Output:

```
"#!/bin/sh\necho \"Starting desk-api...\"\nexec desk-api --port 8080\n"
```

The double-single-quote syntax (`''`) is ideal for inline shell scripts and configuration files.

The trap

Quoting filesystem paths like `"./config.nix"` instead of `./config.nix`. A quoted path is an ordinary string; an unquoted path is a Nix path type that gets copied into `/nix/store` when referenced inside a derivation.

The boring rule

- Use double quotes (`"..."`) for single-line strings; use double single-quotes (`''...''`) for multi-line scripts.
- Use unquoted paths (`./path`) when referring to local files to be packaged.
- Keep data types pure and predictable.

Try this

1. Evaluate `let a = "world"; in "Hello ${a}!"` using `nix eval --expr 'let a = "world"; in "Hello ${a}!"'`.

Let-In Blocks and Local Bindings

Let-In Blocks and Local Bindings

Nix does not have mutable assignment statements. The boring default is: **use `let ... in ...` to introduce clear, locally-scoped, immutable bindings that make complex expressions readable**.

Mental model

The `let ... in` construct has two sections: - `let`: A block of bindings in the form `name = expression;`. - `in`: The expression that is evaluated, in which those bindings are accessible.

Bindings inside a `let` block are mutually recursive by default: a binding can reference any other binding in the same block, regardless of line order.

```
let
  b = a + 10;
  a = 5;
in
b  # evaluates to 15
```

Evaluation order is driven by dependency need, not textual sequence.

Worked examples

Case 1: Structuring Component Configurations

Save as `desk_service.nix`.

```
# desk_service.nix
let
  domain = "internal.desk";
  port = 443;
  protocol = "https";
  serviceUrl = "${protocol}://${domain}:${toString port}";
in
{
  inherit domain port;
  url = serviceUrl;
}
```

Run:

```
nix-instantiate --eval --strict desk_service.nix
```

Output:

```
{ domain = "internal.desk"; port = 443; url = "https://internal.desk:443"; }
```

`inherit domain port;` is shorthand for `domain = domain; port = port;`.

Case 2: Lexical Scope and Shadowing

Save as `scope.nix`. Inner `let` blocks can shadow outer bindings cleanly.

```
# scope.nix
let
  tier = "standard";
in
{
  outerTier = tier;
  overridden =
    let
      tier = "premium";
    in
    tier;
}
```

Run:

```
nix-instantiate --eval --strict scope.nix
```

Output:

```
{ outerTier = "standard"; overridden = "premium"; }
```

Shadowing is local to the expression following `in`.

The trap

Creating massive 200-line `let` blocks containing unrelated helpers. When a `let` block grows too large, readability degrades.

The fix: Group related helpers into separate `.nix` files and bring them in with `import`.

The boring rule

- Use `let ... in` to name intermediate calculations and keep expressions clean.
- Use `inherit` to avoid redundant `key = key;` syntax.
- Remember bindings are evaluated lazily: unused `let` bindings incur zero runtime cost.

Try this

1. In `desk_service.nix`, add an unused binding `unused = throw "never evaluated";` and verify that evaluation still succeeds because of laziness.

Functions and Argument Defaults

Functions and Argument Defaults

Functions in Nix are first-class values that take exactly one argument and return a value. The boring default is: **accept a single attribute set with explicit destructuring, supply sane default values with ?, and use ... only when wrapping extensible APIs**.

Mental model

Nix function syntax uses a colon `:` separating the argument pattern from the function body:

```
x: x + 1
```

For functions requiring multiple parameters, there are two patterns: 1. **Currying:** `x: y: x + y` 2. **Attribute Set Destructuring (The standard):** `{ name, port ? 8080, ... }: ...`

Destructuring allows named parameters in any order, optional defaults (`? default`), and extra attribute tolerance (`...`).

Worked examples

Case 1: Curried Functions vs Destructured Sets

Save as `functions_demo.nix`.

```
# functions_demo.nix
let
  # Curried
  add = x: y: x + y;

  # Destructured attribute set
  formatServer = { host, port ? 80 }: "${host}:${toString port}";
in
{
  sum = add 10 25;
  defaultPort = formatServer { host = "api.desk"; };
  customPort = formatServer { host = "api.desk"; port = 443; };
}
```

Run:

```
nix-instantiate --eval --strict functions_demo.nix
```

Output:

```
{ customPort = "api.desk:443"; defaultPort = "api.desk:80"; sum = 35; }
```

Destructured attribute sets read like named arguments in modern languages.

Case 2: Capturing Full Sets with the `@` Pattern

Save as `capture.nix`. Access both individual fields and the complete input set.

```
# capture.nix
let
  makeProfile = args@{ user, role ? "member", ... }: {
    username = user;
    userRole = role;
    rawInput = args;
  };
in
makeProfile { user = "alice"; team = "desk-ops"; }
```

Run:

```
nix-instantiate --eval --strict capture.nix
```

Output:

```
{ rawInput = { team = "desk-ops"; user = "alice"; }; userRole = "member"; username = "alice"; }
```

`args@` binds the whole attribute set while `{ user, ... }` extracts specific keys.

The trap

Omitting `...` when a function receives configuration from external modules. If an unexpected attribute is passed without `...`, evaluation halts immediately with an "unexpected argument" error.

The fix: Use `...` at the end of parameter sets for open/extensible interfaces (like package and module declarations).

The boring rule

- Always prefer attribute set destructuring (`{ pkgs, lib, ... }:`) over curried functions for configuration code.
- Provide sensible defaults with `? default`.
- Use `inherit` inside functions to assemble result sets cleanly.

Try this

1. In `formatServer`, add an optional boolean `secure ? false` and output `https://` if true, `http://` if false.

Attribute Sets and Recursive Definitions

Attribute Sets and Recursive Definitions

Attribute sets are the workhorse data structure of Nix. The boring default is: **use standard attribute sets, merge them with `//`, access fields with dot-notation (`a.b.c`), and reserve `rec` for simple, acyclic sibling references**.

Mental model

An attribute set is a collection of key-value pairs wrapped in braces:

```
{ key1 = "value1"; key2 = 42; }
```

Key operations: 1. **Dot Access:** `set.key1` 2. **Fallback Operator (`or`):** `set.missingKey or "default"` 3. **Merge Operator (`//`):** `{ a = 1; b = 2; } // { b = 3; c = 4; }` (right-hand side overwrites left) 4. **Nested Syntax:** `desk.database.port = 5432;` is equivalent to `desk = { database = { port = 5432; }; };` 5. **Recursive Attribute Sets (`rec`):** Allows attributes within the set to reference other attributes in the same set.

Worked examples

Case 1: Merging and Fallback Operations

Save as `attr_operations.nix`.

```
# attr_operations.nix
let
  defaultConfig = {
    host = "127.0.0.1";
    port = 8080;
    workers = 4;
  };

  productionOverrides = {
    port = 443;
    workers = 16;
  };

  finalConfig = defaultConfig // productionOverrides;
in
```

```
  {
    inherit finalConfig;
    customTimeout = finalConfig.timeout or 30;
  }
```

Run:

```
nix-instantiate --eval --strict attr_operations.nix
```

Output:

```
{ customTimeout = 30; finalConfig = { host = "127.0.0.1"; port = 443; workers = 16; }; }
```

`productionOverrides` cleanly overrides specific keys while leaving `host` intact.

Case 2: Recursive Attribute Sets (rec)

Save as `rec_demo.nix`.

```
# rec_demo.nix
rec {
  domain = "desk.corp";
  authService = "https://auth.${domain}";
  billingService = "https://billing.${domain}";
  endpoints = [ authService billingService ];
}
```

Run:

```
nix-instantiate --eval --strict rec_demo.nix
```

Output:

```
{ authService = "https://auth.desk.corp"; billingService = "https://billing.desk.corp"; domain
```

Fields dynamically reference `domain` without needing an enclosing `let` block.

The trap

Using `rec` and then overriding a key with `//`. Overriding a field in a `rec` set does not automatically update other fields that referenced the original value.

```
# Anti-pattern: expecting // to update references in rec
let
```

```
  base = rec { x = 1; y = x + 1; };
  updated = base // { x = 10; };
in
updated.y # Still evaluates to 2, not 11!
```

The fix: Use functions (`makeConfig = { x, ... }: ...`) or `let ... in` blocks when you need values to propagate after overrides.

The boring rule

- Use `a.b.c or default` for safe property access.
- Use `//` for shallow configuration overrides.
- Use `rec` only when the set is self-contained and will not be extended or overridden with `//`.

Try this

1. In `attr_operations.nix`, verify what happens if you access `finalConfig.invalid` without `or`.

Conditionals, Lists, and Built-in Operations

Conditionals, Lists, and Built-in Operations

Transforming collections and applying conditional flags are common when configuring systems. The boring default is: **use standard `if ... then ... else ...` expressions, concatenate lists with `++`, and leverage `builtins.map` and `builtins.filter` for pure transformations**.

Mental model

1. **Conditionals:** In Nix, `if` is an expression that yields a value, not a control-flow statement. The `else` clause is mandatory:

   ```
   if condition then valueA else valueB
   ```

2. **Lists:** Space-separated elements wrapped in square brackets:

   ```
   [ 1 "two" ./three (4 + 5) ]
   ```

3. **List Concatenation:** Using `++`:

   ```
   [ 1 2 ] ++ [ 3 4 ] # evaluates to [ 1 2 3 4 ]
   ```

4. **Built-in Pure Operations:** Functions provided by the Nix runtime under `builtins`, including `builtins.map`, `builtins.filter`, `builtins.length`, and `builtins.elem`.

Worked examples

Case 1: Conditional Configuration Flags

Save as `conditional_config.nix`.

```
# conditional_config.nix
let
  isProduction = true;
  replicaCount = if isProduction then 5 else 1;
  logLevel = if isProduction then "warn" else "debug";
in
{
  replicas = replicaCount;
  logging = logLevel;
```

```
}
```

Run:

```
nix-instantiate --eval --strict conditional_config.nix
```

Output:

```
{ logging = "warn"; replicas = 5; }
```

Both branches must return valid expressions; the selected branch is evaluated lazily.

Case 2: Transforming and Filtering Lists

Save as `list_operations.nix`. Filter table numbers and map them into server records.

```
# list_operations.nix
let
  tableNumbers = [ 1 2 3 4 5 6 7 8 9 10 ];
  isPriority = n: n <= 3;
  priorityTables = builtins.filter isPriority tableNumbers;
  deskWorkers = builtins.map (n: { table = n; status = "active"; }) priorityTables;
in
deskWorkers
```

Run:

```
nix-instantiate --eval --strict list_operations.nix
```

Output:

```
[ { status = "active"; table = 1; } { status = "active"; table = 2; } { status = "active"; tabl
```

`builtins.filter` and `builtins.map` provide straightforward functional data processing.

The trap

Forgetting the `else` clause in an `if` expression. Because Nix is purely functional, an `if` expression without an `else` has no defined fallback value and produces a syntax error.

The fix: Always supply both `then` and `else` branches. If you want an attribute to be conditionally omitted, use `lib.optionalAttrs` or `lib.mkIf`.

The boring rule

- Every `if` must have an `else`.
- Use `++` to combine lists.
- Rely on built-ins (`builtins.map`, `builtins.filter`) for standard collection processing.

Try this

1. In `list_operations.nix`, use `builtins.length` to calculate the count of priority tables.

Importing Files and Building Abstractions

Importing Files and Building Abstractions

Real systems cannot fit inside a single `.nix` file. The boring default is: **split logic into small, modular files, pass explicit arguments into them, and import them with the `import` operator**.

Mental model

In Nix, `import` is a built-in function that takes a path to a `.nix` file and evaluates it:

```
import ./constants.nix
```

If the imported file evaluates to a function, you pass arguments immediately:

```
import ./service.nix { inherit pkgs; port = 8080; }
```

There are no circular dependencies, hidden global namespaces, or implicit module registries. An imported file only has access to what is explicitly passed into it.

Worked examples

Case 1: Modular Service Definition

Save as `database.nix`. A reusable database module that takes an environment argument.

```
# database.nix
{ environment }:

let
  isProduction = environment == "production";
in
{
  dbHost = if isProduction then "db.desk.corp" else "127.0.0.1";
  dbPort = 5432;
  poolSize = if isProduction then 50 else 5;
}
```

Save as `app.nix`. Import `database.nix` and pass the required configuration.

```nix
# app.nix
let
  dbConfig = import ./database.nix { environment = "production"; };
in
{
  appName = "desk-billing";
  database = dbConfig;
}
```

Run:

```nix
nix-instantiate --eval --strict app.nix
```

Output:

```
{ appName = "desk-billing"; database = { dbHost = "db.desk.corp"; dbPort = 5432; poolSize = 50;
```

Clean, parameter-driven abstraction with zero global state.

Case 2: Standard Library Utilities (pkgs.lib)

Nixpkgs includes `lib`, an extensive library of functional helpers.

Save as `lib_demo.nix`:

```nix
# lib_demo.nix
let
  # Demonstrating lib helpers conceptually
  optional = condition: element: if condition then [ element ] else [];
  flatten = builtins.concatLists;

  features = flatten [
    [ "core-api" ]
    (optional true "metrics-exporter")
    (optional false "debug-profiler")
  ];
in
features
```

Run:

```nix
nix-instantiate --eval --strict lib_demo.nix
```

Output:

```
[ "core-api" "metrics-exporter" ]
```

Functions like `lib.optional` make conditional list assemblies clean and readable.

The trap

Importing files that rely on ambient path lookups like `<nixpkgs>` in production configurations. `<nixpkgs>` depends on the host machine's `$NIX_PATH` channel, reintroducing non-reproducibility across different machines.

The fix: Pin dependencies using Flakes or explicit pinned tarball URLs (`builtins.fetchTarball`).

The boring rule

- Structure large configurations as directories of focused `.nix` files.
- Each `.nix` file should return a function accepting its dependencies (`{ lib, config, ... }:`).
- Keep file boundaries clear and avoid deeply nested abstractions.

Try this

1. Modify `database.nix` to support a `"staging"` environment that uses port `5433`.

Dev Environments

Nix Shells and the devShell Concept

Nix Shells and the devShell Concept

Development environments rot when developers install tools directly onto their host machines. The boring default is: **define a `shell.nix` or `devShell` declaring the exact tools required, and activate it with `nix-shell` or `nix develop`**.

Mental model

A Nix devShell is an ephemeral execution environment: 1. It downloads or builds only the packages listed in `packages` or `buildInputs`. 2. It sets up `$PATH`, environment variables, and compiler flags in a child shell. 3. It does not alter your workstation's global system directories. 4. When you exit the shell (`exit` or Ctrl+D), your environment reverts instantly to normal.

```
Host OS ($PATH = /usr/bin)

    nix-shell starts
          Injects /nix/store/abc...-go-1.27/bin into $PATH
          Runs shellHook
          Developer compiles and tests

    exit drops back to pristine Host OS ($PATH = /usr/bin)
```

Worked examples

Case 1: Simple shell.nix

Save as `shell.nix`.

```
# shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  packages = [
    pkgs.ripgrep
    pkgs.jq
  ];

  shellHook = ''
    echo "=== Desk Shell Ready ==="
```

```
  '';
}
```

Run:

```
nix-shell --run "echo Shell active"
```

Output:

```
=== Desk Shell Ready ===
Shell active
```

The tools are available only within that session.

Case 2: Setting Environment Variables in a Shell

Save as `env_shell.nix`.

```
# env_shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  packages = [ pkgs.curl ];

  env = {
    DESK_API_URL = "https://api.desk.internal";
    DESK_TIMEOUT = "30";
  };

  shellHook = ''
    echo "Targeting API: $DESK_API_URL"
  '';
}
```

Run:

```
nix-shell env_shell.nix --run "echo Timeout: \$DESK_TIMEOUT"
```

Output:

```
Targeting API: https://api.desk.internal
Timeout: 30
```

Environment variables are isolated and declared in code.

The trap

Installing development compilers like Go, Rust, or Node globally with system package managers (`apt`, `brew`). Different projects inevitably require different versions, leading to version conflicts.

The fix: Use a `shell.nix` in each repository.

The boring rule

- Every project repository must have a root `shell.nix` or `flake.nix`.
- Use `pkgs.mkShell` with `packages` for modern shells.
- Use `shellHook` for local environment configuration and diagnostics.

Try this

1. Add `pkgs.tree` to `shell.nix` and run `nix-shell --run "tree -L 1"`.

Multi-Language Development Environments

Multi-Language Development Environments

Modern systems frequently combine multiple languages: a Go backend, a TypeScript frontend, Python data scripts, and C shared libraries. The boring default is: **declare all runtimes, compilers, and library headers inside a single unified `mkShell` definition**.

Mental model

In traditional systems, managing polyglot stacks requires coordinating multiple independent version managers (`asdf`, `nvm`, `pyenv`, `rustup`) and installing system development packages (`libssl-dev`, `libpq-dev`).

Nix unifies this: - Compilers (`go`, `rustc`, `nodejs`) are just packages. - C libraries (`openssl`, `postgresql`) provide headers and shared objects automatically. - `pkg-config` is wired up automatically when included in `nativeBuildInputs`.

Worked examples

Case 1: Unified Polyglot Stack

Save as `polyglot.nix`.

```
# polyglot.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  packages = [
    pkgs.go
    pkgs.nodejs_22
    (pkgs.python3.withPackages (ps: [ ps.requests ps.pydantic ]))
    pkgs.sqlite
  ];

  shellHook = ''
    echo "Polyglot environment loaded: Go, Node, Python, SQLite"
  '';
}
```

Run:

```
nix-shell polyglot.nix --run "go version && node --version && python3 --version"
```

Output:

```
Polyglot environment loaded: Go, Node, Python, SQLite
go version go1.27.1 linux/amd64
v22.11.0
Python 3.12.7
```

All runtimes coexist without path or library collision.

Case 2: C Libraries and pkg-config

Save as `native_c.nix`. A shell that configures C headers and libraries for applications that compile native extensions.

```
# native_c.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  nativeBuildInputs = [ pkgs.pkg-config ];
  buildInputs = [ pkgs.openssl pkgs.zlib ];

  shellHook = ''
    echo "OpenSSL cflags: $(pkg-config --cflags openssl)"
  '';
}
```

Run:

```
nix-shell native_c.nix --run "echo C libraries configured"
```

Output:

```
OpenSSL cflags: -I/nix/store/...-openssl-3.3.2-dev/include
C libraries configured
```

`pkg-config` points directly to the isolated store header directories.

The trap

Relying on host system header files like `/usr/include/openssl`. If a teammate is on a different Linux distribution or macOS, paths differ and builds fail.

The fix: Always include header and library dependencies explicitly in `buildInputs`.

The boring rule

- Group build-time tools (like `pkg-config`, `cmake`) in `nativeBuildInputs`.
- Group runtime libraries and headers (like `openssl`, `zlib`) in `buildInputs`.
- Use language-specific package wrappers (like `python3.withPackages`).

Try this

1. In `polyglot.nix`, add `pkgs.cargo` and test running `cargo --version`.

Flakes for Reproducible Dev Environments

Flakes for Reproducible Dev Environments

Classic `shell.nix` still relies on `<nixpkgs>`, which can vary across developer machines depending on their channel state. The boring default is: **use modern Nix Flakes (`flake.nix` + `flake.lock`) to pin the exact commit of nixpkgs across the entire team**.

Mental model

Nix Flakes introduce three critical improvements: 1. **Pinned Lockfiles (`flake.lock`):** Automatically records the exact Git commit hash of every dependency. 2. **Standard Output Schema:** Predictable structure (`packages.${system}`, `devShells.${system}.default`). 3. **Pure Evaluation:** Flakes disallow reading undeclared files, ambient environment variables, or mutable network channels.

```
flake.nix (declares inputs & outputs)


flake.lock (records exact Git commit of nixpkgs)


nix develop (creates identical shell on all machines)
```

Worked examples

Case 1: Minimal Flake Development Shell

Save as `flake.nix`.

```
# flake.nix
{
  description = "Desk management service development shell";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
  };

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
```

```
      pkgs = import nixpkgs { inherit system; };
    in
    {
      devShells.${system}.default = pkgs.mkShell {
        packages = [
          pkgs.go
          pkgs.gopls
          pkgs.golangci-lint
        ];

        shellHook = ''
          echo "=== Pinned Go 1.27 Dev Environment ==="
        '';
      };
    };
}
```

Run:

```
nix develop --command go version
```

Output:

```
=== Pinned Go 1.27 Dev Environment ===
go version go1.27.1 linux/amd64
```

The exact commit of `nixpkgs` is frozen into `flake.lock`.

Case 2: Updating Dependencies

When you are ready to update packages across the team, update the lockfile explicitly:

Run:

```
nix flake update
```

Output:

```
updating lock file '/home/user/project/flake.lock':
  • Updated input 'nixpkgs':
    'github:NixOS/nixpkgs/...' (2026-09-01)
  → 'github:NixOS/nixpkgs/...' (2026-09-08)
```

Updates are an intentional Git commit, never an accidental surprise.

The trap

Forgetting to check `flake.lock` into Git. Without `flake.lock`, teammates generate their own lockfiles on different dates, losing strict bit-for-bit reproducibility.

The fix: Always commit `flake.lock` alongside `flake.nix`.

The boring rule

- Always use Flakes for new development environments.
- Always commit `flake.lock` to source control.
- Use `nix develop` to enter Flake-based development shells.

Try this

1. Inspect `flake.lock` with `cat flake.lock` or `jq . flake.lock` and locate the `rev` (revision) field.

Environment Management with Home Manager

Environment Management with Home Manager

Project-specific devShells provide tools for a specific repository, but developers also need personal utilities across all shells (Git configurations, shell prompts, editor settings). The boring default is: **use Home Manager to manage user-level tools and configuration files declaratively**.

Mental model

Home Manager applies the Nix module system to the user's home directory (`$HOME`): 1. **User Packages (`home.packages`):** Installs CLI tools into `~/.nix-profile` without requiring root permissions. 2. **Managed Dotfiles:** Generates read-only symlinks in `~/.config/` pointing to `/nix/store` paths. 3. **Integration with devShells:** Personal utilities (like your favorite shell prompt, tmux bindings, and Git aliases) remain active globally, while project-specific compilers and libraries are injected on top via `devShell`.

```
User Space ($HOME)
     Managed by Home Manager (git, zsh, vim, starship prompt)
     Inside Project Directory:
           Activated devShell adds project Go 1.27 and Postgres CLI
```

Worked examples

Case 1: Minimal home.nix Configuration

Save as `home.nix`.

```
# home.nix
{ pkgs, ... }:

{
  home.username = "deskuser";
  home.homeDirectory = "/home/deskuser";
  home.stateVersion = "24.11";

  home.packages = [
    pkgs.htop
    pkgs.bat
    pkgs.fzf
  ];
```

```
  programs.git = {
    enable = true;
    userName = "Desk Engineer";
    userEmail = "engineer@desk.corp";
  };
}
```

Run:

```
home-manager switch
```

Output:

```
Starting Home Manager activation
Creating symlinks in /home/deskuser...
Activating git configuration...
Home Manager switch complete!
```

Git configuration and personal utilities are synchronized declaratively.

The trap

Using Home Manager to install project-specific language runtimes (e.g., globally pinning Go 1.27 in `home.packages`). When another project requires Go 1.26, global conflicts return.

The fix: Reserve Home Manager for personal user utilities. Keep compilers and language libraries inside project `devShells`.

The boring rule

- Home Manager manages user preferences and personal CLI tools.
- Project repositories manage their own compilers and runtimes via `devShell`.
- Keep dotfiles version-controlled in Git.

Try this

1. In `home.nix`, configure `programs.bash.enable = true;` with a custom shell alias `ll = "ls -la";`.

Migrating from Docker Desktop to Nix DevShells

Migrating from Docker Desktop to Nix DevShells

Many teams default to running basic development tools inside Docker containers to avoid dependency drift. The boring default is: **migrate local development toolchains to native Nix `devShells` for near-zero startup latency, direct editor integration, and fractional resource consumption**.

Mental model

Dimension	Docker Desktop Container	Nix devShell
Architecture	Linux VM + Container Daemon	Native Host Process
Startup Latency	5 – 20 seconds	$<$ 50 milliseconds
File I/O Performance	Slow across VM bind-mounts	Native host filesystem speed
IDE Integration	Requires Remote-Containers plugins	Direct local LSP & debugger integration
Resource Usage	4 – 8 GB RAM continuously	Exact binary memory only when executed

Containers remain the right choice for isolated services (e.g. running an actual PostgreSQL database or Redis instance). They are the wrong choice for running simple compilers (`go`, `rustc`, `npm`).

Worked examples

Case 1: The Docker Compose Pattern vs The Nix devShell Pattern

Compare a typical `docker-compose.yml` with a clean `flake.nix`.

Docker setup requires bind mounts, ports, and a persistent background VM.

The Nix equivalent:

Save as `flake.nix`:

```
# flake.nix
{
  description = "Fast native desk development environment";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
```

```
  outputs = { self, nixpkgs }:
    let
      pkgs = import nixpkgs { system = "x86_64-linux"; };
    in
    {
      devShells.x86_64-linux.default = pkgs.mkShell {
        packages = [
          pkgs.go
          pkgs.golangci-lint
          pkgs.delve
        ];
      };
    };
}
```

Run:

```
nix develop --command go test ./...
```

Output:

```
ok      desk/service    0.015s
```

Compilation and tests run at native host speed without VM translation layers or volume-mount caching slowdowns.

Case 2: Hybrid Workflow — Native Tooling with Containerized Backends

The recommended production pattern: - **Language toolchains & linters:** Run natively via Nix `devShell`. - **Stateful databases & caches:** Run as lightweight containers via Podman or Docker.

Save as `start_services.sh`:

```
# start_services.sh
#!/usr/bin/env bash
set -euo pipefail

echo "Launching isolated database container..."
podman run -d --name desk-db -p 5432:5432 -e POSTGRES_PASSWORD=secret postgres:16-alpine ||

echo "Running migrations with native Nix-provided toolchain:"
psql -h 127.0.0.1 -U postgres -c "SELECT 'Desk database online' AS status;"
```

Run:

```
nix-shell -p postgresql --run "bash start_services.sh"
```

Output:

```
Launching isolated database container...
Running migrations with native Nix-provided toolchain:
        status
----------------------
 Desk database online
(1 row)
```

The CLI tools are provided instantly by Nix; the database runs isolated in a container.

The trap

Running full GUI editors or IDEs inside containers or remote VMs when simple local toolchain isolation is all that was needed.

The fix: Use Nix to provide the compilers and tools on your host OS. Your editor connects directly to native binaries on `$PATH`.

The boring rule

- Use Nix `devShells` for compilers, language servers, CLI tools, and linters.
- Reserve containers for long-running stateful services (databases, message queues).
- Enjoy native host file I/O speed and seamless editor integration.

Try this

1. Compare the time taken to run `go version` natively in a Nix devShell vs inside a `docker run --rm golang go version` container.